ORBIT PLATFORM · TECHNICAL REFERENCE

Backoffice Web Platform

The operations console. Live dispatch, compliance, finance and platform controls — built for people who are dealing with something going wrong.

Repository	orbit-backoffice-web
Kind	Client
Ports	5174
Tests	23 passing
Platform	React 19

Generated from the service source. Regenerate rather than editing.

Contents

Overview

`orbit-backoffice-web` is the operations console: live dispatch, compliance, finance and platform controls.

Same stack as the enterprise console — React 19, TypeScript, Tailwind 4, TanStack, Vite, pnpm — and the same shared foundations. A different job, though: this one is used by people who are dealing with something going wrong.

Responsibilities

- The live-ops wall display
- Ride search and the ride detail escalation screen
- Driver KYC review and the fraud queue
- Ledger inspection and the four-eyes approvals queue
- Surge kill-switch and the audit log

Non-responsibilities

- **Deciding anything.** Every privileged action goes to the admin BFF, which audits it
- **Owning data.** Every screen is a live read

Shared foundations

The API client, session handling, problem-details mapping, query client, query keys and the UI primitives are **the same code** as `orbit-enterprise-web`. See that service's technical reference for:

- Why every call goes through the gateway and nothing holds a service URL
- Why the access token lives in memory and refreshes collapse into one request
- Why the UI branches on `code` rather than `detail`
- Why a 409 is not retried and mutations are never retried automatically
- Why money stays an integer until `<Money>` renders it
- The container, nginx and pnpm configuration

This document covers what is different.

Dark by default

The enterprise console defaults to light and follows the system. This one defaults to **dark**.

> WHY Live-ops screens sit on wall displays in rooms that stay dim for hours. A white canvas at 3 a.m. is the reason people turn the monitor away, and a monitor turned away is a dashboard nobody is watching.

The toggle still works in both directions, and the choice persists.

Grouped navigation

```
Operations   Live ops · Rides
Compliance   KYC queue · Fraud alerts
Finance      Ledger · Refunds
Platform     Surge & zones · Audit log
```

DECISION The console spans four separate jobs, and the people doing them are usually different people. A flat list of nine links makes everyone scan past six things they never touch, every time.

Live ops polls; it does not hold a socket

Five-second polling, `staleTime: 0`.

> **DECISION** The gateway's WebSocket capacity is sized for riders and drivers — hundreds of thousands of them. Spending connections on a handful of ops screens that are happy with five-second-old numbers is the wrong trade.

`staleTime` is zero because this screen is only ever looked at for the **current** number. A cached snapshot on a wall display is worse than a blank one: nobody can tell it is old.

A failure says so loudly

```
{isError
  ? <span className="text-fg-danger">Not updating — last figures may be stale</span>
  : `Updated ${new Date(dataUpdatedAt).toLocaleTimeString('en-NG')}`}
```

> **WARNING** A frozen dashboard that still looks healthy is how a room spends twenty minutes acting on numbers from before the incident. The timestamp is always visible so the staleness is readable even when polling is working.

Thresholds change colour

TILE	TURNS
Searching	Danger, when searching rides exceed idle drivers
Median match	Warning, above the §3 target of 20 s

> **WHY** The number that matters should be the one that changes colour, rather than one buried in a dashboard nobody reads. A tile that is permanently amber stops meaning anything, which is why the enterprise console's pending-approvals tile is only coloured when there is something to act on.

The ride detail screen

One escalation lives here, and it is assembled by the admin BFF from five services in a single call.

Sections that fail are named, not fatal

```
{data.unavailableSections.length > 0 && (
  <p role="status">Could not load: {sections.join(', ')}. Everything else here is current.
</p>
)}
```

> WHY A support agent looking at a ride during a payment outage still needs the ride, the rider and the driver. A page that fails wholesale because one upstream is slow is unavailable exactly when a rider is on the phone asking about their fare.

A genuinely absent section — a ride with no payment yet — renders as "No payment yet" rather than as an outage.

Arrears is presented as a normal state

```
In arrears. The ride completed; the debt is recorded.
```

> DECISION Not an error, and not styled as one. The rider travelled and the money did not follow — payment failure never blocks the physical service. An agent who reads it as a fault starts investigating something that is working as designed.

An unmatched trace is flagged

A trace with zero match confidence shows a warning: the raw distance was billed, and it should be treated with caution in a dispute.

Force-cancel

The panel is **not rendered at all** on a terminal ride, rather than rendered and refused.

The reason field requires ten characters, with a live countdown of how many are still needed.

> **NOTE** The client-side minimum is a **courtesy, not the control**. The control is in the BFF's aggregate, and a client that skips this check still cannot get past it. Showing the requirement before submission just avoids a round trip that ends in a validation error.

A 409 renders as "The ride changed while you were typing. Reload and check its current state" — because that is what happened, and it is a normal outcome when an agent and a driver act at the same moment.

The audit log

Read-only. There is no route in this console that writes or deletes a record.

> **WHY** An endpoint that could add one by hand would make every record in the table arguable.

A resource is **required** before anything is fetched (`enabled: resource.length > 3`).

> **WHY** The audit table is large, and "show me everything" is a rather different question from the one anybody means to ask.

A **failed** privileged action is displayed as prominently as a successful one, with a danger border. Somebody tried to force a state transition; that it did not work is not a reason to let it blend into the successes.

Fraud alerts

Every case renders its contributing features with the plain-language description.

```
<span className="tabular">{(feature.contribution * 100).toFixed(0)}%</span>
<span>{feature.description}</span>
```

WHY A decision that cannot be explained cannot be appealed, and an unappealable automated suspension is exactly the failure mode the fraud subsystem exists to prevent. These are the words a reviewer reads aloud to a driver.

The dismiss button is labelled **"Dismiss and lift"**, because that is what it does.

DECISION A queue where dismissing leaves the punishment in place looks like due process and delivers none. Naming the effect in the button is the difference between a reviewer who knows what they are doing and one who finds out later.

The ledger

Read-only, and **debits and credits sit in separate columns**.

> WHY That is the way a ledger is read. One signed column forces the reader to parse a minus sign on every row to work out which way the money went, on a screen whose entire purpose is scanning.

There is no route that writes to it. Balances are derived from postings, never stored, and a screen that could adjust one directly would break the property the whole payments subsystem rests on.

Approvals

The queue refetches every 30 seconds and **excludes lapsed requests**.

> WHY Showing an approver something they cannot action is how a queue loses their attention, and a queue nobody reads is not a control.

A self-approval refusal renders as "You raised this request, so somebody else has to decide it" — being refused with no explanation after typing a justification is a poor way to learn a rule.

An expired request renders as "That request expired. It has to be raised again", which is actionable, rather than a generic conflict.

Surge controls

Every change goes to the four-eyes queue, and the UI says so **before** the operator commits.

```
{pending.engage
  ? 'This will be sent for a second approval.'
  : 'Restoring surge is also sent for approval.'}
```

DECISION Restoring surge is called out as loudly as suppressing it. Turning it back on in a city is at least as consequential, and only one of the two is ever anticipated — so the one nobody expects is the one the UI has to be explicit about.

The success message is deliberately precise:

```
Sent for approval. Nothing has changed yet — surge is still running in that zone.
```

An operator who reads "done" and walks away has not done the thing they think they have.

Routing

```
/                       Live ops
/rides                  Search
/ride/$rideId           Detail
/compliance             KYC queue
/kyc/$driverId          Driver detail
/fraud · /ledger · /refunds · /surge · /audit
```

The two detail routes sit on **their own path segments** rather than under their list pages.

> NOTE A nav link to `/compliance` next to a route at `/compliance/drivers/$driverId` makes the router treat the nav link as possibly needing a `driverId` — which is both a type error and a fair description of the ambiguity. Moving the detail routes to `/kyc/$id` and `/ride/$id` removes it entirely.

The deep link survives the auth redirect

```
throw redirect({ to: '/sign-in', search: { redirect: location.href } })
```

> WHY A support agent pasted a ride URL into a chat. Landing the recipient on a dashboard after signing in loses the thing they were sent, and they have to ask for it again.

The redirect target is read from the **URL**, not from the router's typed search, because it is an arbitrary path the router cannot type as one of its known routes — and it is navigated to as `href` rather than `to` for the same reason.

> WARNING It is checked for same-origin, and the `//` check matters: `//evil.example` is a protocol-relative URL, not a path. Without it the sign-in page is an **open redirect** — a link that authenticates the user and then hands them to somebody else's site, with the whole flow looking entirely legitimate.

Running it

```
cp .env.example .env.local
pnpm install
pnpm dev
```

http://localhost:5174.

COMMAND	
`pnpm dev` · `pnpm build` · `pnpm test` · `pnpm lint`	As the enterprise console

Testing

23 tests, shared with the enterprise console where the code is shared: the API error taxonomy, the session's refresh collapsing and storage discipline, and the UI primitives.

> NOTE The shared code is genuinely the same code rather than a copy that has drifted. A bug found in one console's session handling is a bug fixed in both, and the tests are duplicated deliberately so that either repository fails on its own if the shared behaviour regresses there.